

Facultad de Ingeniería, Universidad de Buenos Aires

ARQUITECTURA DE SOFTWARE (75.73/TB034)

Trabajo Práctico N°1

Primer cuatrimestre 2026

Alumnos

Nombre	Padrón	E-mail
Avalos, Victoria Belén	108434	vavalos@fi.uba.ar
Aldonate, Lucas	100030	laldonate@fi.uba.ar
Paredes Tavera, Rogger Aldair	97976	rparedest@fi.uba.ar
Castaño, Mateo	110114	macastano@fi.uba.ar
Bragantini, Franco Julian	97190	fbragantini@fi.uba.ar

Mayo de 2026

Índice

1. Introducción	2
2. Análisis del sistema actual	3
3. Atributos de Calidad claves	4
3.1. Availability	4
3.2. Performance	4
3.3. Consistencia / Integridad de datos	4
3.4. Scalability	5
3.5. Visibility	5
4. Otros Atributos de Calidad	6
4.1. Observaciones generales	6
5. Implementación de Métricas de Negocio	7
5.1. Escenario de Prueba y Emisión de Métricas	7
5.2. Visualización en Grafana	7
5.3. Resultados Obtenidos	8
6. Problemas Identificados y Soluciones Propuestas	9
6.1. Modularizar la lógica en capas	11
6.1.1. Hipótesis inicial	11
6.1.2. Resultado obtenido	11
6.1.3. Conclusiones del cambio	14
6.2. Migrar el estado a Redis	15
6.2.1. Hipótesis inicial	15
6.2.2. Resultado obtenido — escenario rates (solo lectura)	15
6.2.3. Análisis cualitativo y trade-offs	16
6.3. Instanciar múltiples instancias del backend	18
6.3.1. Hipótesis inicial	18
6.3.2. Cambios	18
6.3.3. Escenarios	19
6.3.4. Conclusiones	21
6.3.5. Diagrama	22
6.4. Limitar cantidad, tamaño de request y tiempo de respuesta	23
6.4.1. Escenario 1: Arquitectura base	23
6.4.2. Escenario 2: Arquitectura mejorada con NGINX	24
6.4.3. Resultado obtenido	25
6.4.4. Conclusión	25
6.5. Versión Final del Sistema	26
7. Mediciones Finales y Observabilidad	27
7.1. Análisis de resultados	27
8. Conclusiones	29

1. Introducción

En el presente trabajo se realiza un análisis de la arquitectura, el código y la infraestructura del servicio de cambio de monedas provisto por arVault, una fintech en crecimiento que busca mejorar la calidad de su plataforma ante problemas reportados por sus usuarios y la necesidad de atraer nuevas inversiones.

El objetivo principal es evaluar el estado actual del sistema desde la perspectiva de los atributos de calidad, identificando cuáles son los más relevantes en este contexto y cómo las decisiones de diseño existentes impactan sobre ellos. A partir de este análisis, se busca detectar limitaciones, riesgos y oportunidades de mejora en la solución actual.

Para complementar el análisis teórico, se realizarán pruebas de carga sobre el sistema, recolectando y visualizando métricas que permitan observar su comportamiento bajo distintos escenarios. Estas mediciones servirán como base para proponer e implementar modificaciones orientadas a mejorar atributos de calidad específicos, evaluando su impacto mediante evidencia cuantitativa y analizando los trade-offs involucrados.

Asimismo, se elaborarán diagramas de tipo Components & Connectors que representen tanto la arquitectura original como las variantes propuestas, con el fin de reflejar de manera clara las decisiones de diseño y sus implicancias.

Finalmente, el trabajo busca no solo mejorar el sistema existente, sino también justificar las decisiones adoptadas desde una perspectiva arquitectónica, considerando tanto los requerimientos técnicos como las necesidades del negocio.

2. Análisis del sistema actual

El sistema está compuesto por varios componentes integrados en una solución contenerizada. En el borde se sitúa un proxy HTTP que recibe el tráfico externo y enruta las peticiones hacia la API; se asume que la autenticación y la autorización son delegadas al servicio vaultSec, por lo que las solicitudes que llegan a la API ya están autenticadas y autorizadas.

La API es un servicio monolítico escrito en Node.js/Express que expone los endpoints públicos para la operación de cambio de monedas y orquesta la lógica de negocio: validación de la operación, coordinación de transferencias y actualización del estado de cuentas y tasas. La gestión de estado se mantiene en memoria en tiempo de ejecución y se persiste periódicamente a un almacenamiento local; además existe un componente lógico de registro (log) que guarda el historial de operaciones.

Para las operaciones de movimiento de fondos la API delega en un servicio de transferencia externo simplificado que introduce una latencia de entre 200 y 400 milisegundos. La plataforma de observabilidad está formada por agentes de métricas y una pila de recolección/visualización (métricas expuestas por el entorno, agregación y almacenamiento temporal, y dashboards de visualización) que se usa para analizar rendimiento y negocio.

A continuación se presenta un diagrama de componentes y conectores de la arquitectura del sistema actual.

Vista de Componentes y Conectores

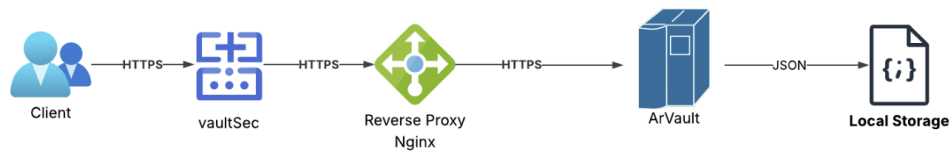


Figura 1: Diagrama de Components & Connectors

En las siguientes secciones se realizará un análisis del código, infraestructura y arquitectura del sistema y cómo se encuentran afectados los distintos atributos de calidad. Primero se identificarán cuáles son los atributos de calidad clave para este servicio, y luego se estudiará cómo se ven influenciados tanto los atributos clave como los de menor importancia.

3. Atributos de Calidad claves

Teniendo en cuenta el tipo de servicio, el contexto de las fintechs y el estado actual del código, infraestructura y arquitectura del sistema, se identificaron los siguientes atributos de calidad clave comenzando con los más relevantes.

3.1. Availability

En cuanto a disponibilidad, una fintech que maneja dinero no puede permitirse caídas. Actualmente, la API corre en un único contenedor sin réplicas, lo que constituye un single point of failure: cualquier fallo del proceso o del nodo vuelve el servicio completamente inaccesible.

Además, no se observan mecanismos de resiliencia como health checks o políticas de reinicio automático. El uso de almacenamiento local dentro del contenedor impide escalar horizontalmente sin riesgo de inconsistencias, ya que no existe un estado compartido entre instancias.

Si bien el sistema utiliza Nginx como punto de entrada, esto no mejora la disponibilidad en el estado actual, dado que no hay múltiples instancias a las cuales balancear tráfico. No obstante, su presencia facilita una futura incorporación de réplicas mediante configuración de upstreams.

En el contexto de una billetera virtual, la indisponibilidad impacta directamente en la operatoria de los usuarios, generando pérdidas económicas, daño reputacional y disminución en la confianza de potenciales inversores.

3.2. Performance

Los reclamos de los usuarios apuntan justamente a problemas de uso. Este atributo es fundamental para una empresa fintech, ya que impacta directamente en la experiencia de usuario y en la percepción de confiabilidad del servicio.

En el sistema existen cuellos de botella concretos. La operación de cambio de moneda realiza dos invocaciones secuenciales al servicio externo de transferencias, cada una con una latencia de entre 200 y 400 milisegundos (según lo definido en el enunciado). Esto introduce un tiempo mínimo de respuesta de entre 400 y 800 milisegundos por operación, afectando principalmente la latencia.

Adicionalmente, en state.js se serializa periódicamente el log completo a disco mediante `JSON.stringify(log, null, 2)`, una operación de complejidad $O(n)$ que crece con el tamaño del log. Esta tarea puede bloquear el event loop de Node.js, degradando tanto la latencia como el throughput bajo carga.

En escenarios de alta concurrencia, estas decisiones pueden generar acumulación de requests, aumento en los tiempos de respuesta y potenciales fallos por timeout.

3.3. Consistencia / Integridad de datos

La consistencia e integridad de los datos son atributos críticos en este sistema y actualmente presentan debilidades significativas.

Existen condiciones de carrera en la operación de intercambio (`exchange()`): múltiples requests concurrentes pueden leer y validar balances antes de que se apliquen las actualizaciones, lo que habilita escenarios de sobregiro.

Además, el estado se mantiene en memoria y se persiste de forma diferida (hasta 1 segundo después), lo que implica que ante un fallo del proceso se pueden perder operaciones ya confirmadas lógicamente, generando inconsistencias entre el estado real y el registro de operaciones.

Por último, los mecanismos de compensación no son atómicos ni transaccionales, y asumen implícitamente que los rollbacks siempre son exitosos, lo cual no está garantizado.

En una billetera virtual, este tipo de inconsistencias puede derivar en pérdidas económicas directas, por lo que este atributo resulta prioritario.

3.4. Scalability

El sistema actual no permite escalar horizontalmente debido a que el estado se encuentra almacenado en memoria dentro de un único proceso y persistido en archivos locales del contenedor.

Replicar la aplicación en este contexto generaría múltiples instancias con estados divergentes, lo que rompe la consistencia del sistema. En consecuencia, la escalabilidad queda limitada a un único nodo (escalado vertical).

Para habilitar escalabilidad horizontal, sería necesario introducir un mecanismo de almacenamiento compartido (por ejemplo, una base de datos o un sistema como Redis).

Dado que arVault busca crecer en cantidad de usuarios, esta limitación representa un riesgo importante a mediano plazo.

3.5. Visibility

Actualmente, el sistema cuenta con un stack de métricas (cAdvisor, StatsD, Graphite y Grafana), pero la aplicación no emite métricas propias, lo que limita significativamente la observabilidad.

No se registran métricas clave como cantidad de requests, tasas de error o latencias por endpoint. Tampoco se mide información crítica para el negocio, como el volumen operado y el neto por moneda a lo largo del tiempo.

Esta falta de visibilidad dificulta la detección temprana de problemas, el diagnóstico de fallos y la toma de decisiones basada en datos.

Este atributo impacta directamente en la disponibilidad y la performance, ya que sin métricas adecuadas resulta complejo identificar cuellos de botella o incidentes de forma oportuna.

4. Otros Atributos de Calidad

- **Testability:** El sistema actualmente no cuenta con tests automatizados. Si bien el diseño modular mediante exports facilita la escritura de unit tests (por ejemplo sobre `exchange()` o las funciones de estado), la falta de separación clara de responsabilidades y el uso de estado global dificultan el aislamiento de componentes para pruebas.

Por otro lado, al tratarse de una API REST, los endpoints pueden probarse fácilmente mediante herramientas como Postman o Artillery (de hecho, se dispone de una colección de Postman), lo que favorece el testing manual e integración.

- **Portability:** La aplicación se encuentra containerizada mediante Docker, lo que favorece la portabilidad al permitir su despliegue en cualquier entorno compatible sin dependencias adicionales.
- **Interoperability:** El hecho de que el sistema exponga una API REST y cuente con una colección de Postman facilita su integración con otros sistemas y clientes.
- **Usability:** Al no contar con una interfaz de usuario propia, la experiencia de usuario final depende de aplicaciones externas. En cuanto a la API, los endpoints resultan relativamente simples y comprensibles, lo que facilita su uso desde clientes.
- **Manageability:** El sistema presenta limitaciones en términos de operación: cuenta con logging mínimo, no posee health checks ni políticas de reinicio automático, y carece de métricas propias. Esto dificulta la detección, diagnóstico y recuperación ante fallos en entornos productivos.
- **Security:** Los aspectos de autenticación y autorización se encuentran delegados a un componente externo (vaultSec), por lo que las requests que llegan a la API ya se consideran válidas en ese sentido. Sin embargo, el servicio no implementa mecanismos adicionales de seguridad a nivel aplicación, como rate limiting o protección ante abuso, lo que podría representar riesgos ante posibles ataques de denegación de servicios (DoS).
- **Simplicity:** El código es pequeño y directo, lo cual facilita su comprensión. Sin embargo, esta simplicidad se logra a costa de sacrificar robustez y preparación para escenarios productivos.
- **Modifiability:** El código exporta funciones reutilizables, lo que facilita ciertas tareas de refactorización y extensión. Sin embargo, la ausencia de una arquitectura por capas y el acoplamiento entre lógica de negocio y manejo de estado dificultan la introducción de cambios estructurales, como la incorporación de una base de datos.

Además, el uso de variables globales mutables (accounts, rates, log) y la dependencia implícita del orden de inicialización aumentan la complejidad de realizar modificaciones sin introducir errores.

4.1. Observaciones generales

En términos generales, el sistema prioriza simplicidad y rapidez de desarrollo por sobre robustez, lo cual es consistente con la estrategia inicial de la startup (“fake it until you make it”).

Sin embargo, esta decisión impacta negativamente en atributos críticos como disponibilidad, consistencia y escalabilidad, volviendo al sistema inadecuado para un entorno productivo con alta demanda y requisitos de confiabilidad.

En su estado actual, la arquitectura presenta limitaciones estructurales que deberán ser abordadas para sostener el crecimiento del negocio y mejorar la calidad del servicio.

5. Implementación de Métricas de Negocio

Atendiendo a la necesidad de analizar el servicio desde una perspectiva funcional y comercial, se requirió la incorporación de métricas que reflejen el comportamiento de las divisas en la plataforma a lo largo del tiempo. Específicamente, se implementaron métricas para observar el **volumen operado** en cada moneda (donde tanto las compras como las ventas suman al total) y el **neto** (donde las compras suman y las ventas restan).

5.1. Escenario de Prueba y Emisión de Métricas

Para verificar la correcta recolección y agregación de estos datos, se diseñó un escenario de pruebas de carga (`business-metrics-multicurrency.yaml`) utilizando Artillery. El escenario simuló un tráfico constante de negocio durante 120 segundos, con una tasa de llegada de 2 requests por segundo. Se configuraron distintos flujos de intercambio ponderados para simular un comportamiento realista de los usuarios:

- Cambio de ARS a USD (Monto: 500000, Peso: 3)
- Cambio de USD a ARS (Monto: 100, Peso: 2)
- Cambio de EUR a ARS (Monto: 50, Peso: 2)
- Cambio de BRL a ARS (Monto: 200, Peso: 2)

Mediante la integración con el plugin de StatsD, la API fue instrumentada para emitir indicadores de tipo *gauge* hacia Graphite, alojando la información bajo las rutas `stats.gauges.exchange.business.volume.*` y `stats.gauges.exchange.business.net.*`.

5.2. Visualización en Grafana

Con el objetivo de hacer esta información accesible, se construyó un dashboard en Grafana denominado "**Dashboard Negocio - Volumen y Neto**". Este tablero se encarga de consumir la información almacenada en Graphite y consta de múltiples paneles que permiten monitorear el negocio en tiempo real.

A continuación, se presentan las capturas del dashboard diseñado. En la Figura 2 se observa el panel principal comparativo que muestra la evolución histórica del volumen y el neto de todas las monedas en simultáneo.



Figura 2: Dashboard en Grafana mostrando las métricas unificadas comparativas de Volumen y Neto para todas las monedas.

Por otro lado, para evitar el ruido visual ante grandes diferencias de escala entre monedas, se implementaron paneles individuales específicos para observar en detalle el comportamiento de cada divisa por separado, tal como se ilustra en la Figura 3.



Figura 3: Vista detallada de los paneles individuales por moneda en el Dashboard de Grafana.

5.3. Resultados Obtenidos

La ejecución de la simulación de carga arrojó resultados que confirman la estabilidad de la instrumentación bajo tráfico continuo. En los 2 minutos que duró la prueba, se crearon y completaron 240 sesiones de usuario virtuales (VUs).

```
http.codes.200: 240
http.request_rate: 2/sec
http.requests: 240
http.response_time:
  min: 410 ms
  max: 784 ms
  mean: 606.5 ms
  median: 608 ms
  p95: 742.6 ms
  p99: 772.9 ms
```

Como se observa en el reporte de Artillery, el 100 % de las peticiones finalizó de manera exitosa (`http.codes.200: 240`), sin registrarse fallos.

Estos resultados demuestran que la incorporación de la lógica de negocio para emitir métricas analíticas cumplió con los requerimientos de visibilidad solicitados, sin comprometer la disponibilidad ni introducir bloqueos en el flujo de intercambio de la API.

6. Problemas Identificados y Soluciones Propuestas

En esta sección se identificarán los problemas más relevantes que afectan a los atributos de calidad clave definidos previamente, y se propondrán y desarrollarán soluciones a los mismos.

Empezamos corriendo el escenario de la cátedra en la rama base, que le pega a `/rates` continuamente.

```
All VUs finished. Total time: 1 minute, 31 seconds

-----
Summary report @ 18:22:42(-0300)
-----

http.codes.200: ..... 390
http.downloaded_bytes: ..... 42510
http.request_rate: ..... 3/sec
http.requests: ..... 390
http.response_time:
  min: ..... 0
  max: ..... 16
  mean: ..... 1.7
  median: ..... 2
  p95: ..... 3
  p99: ..... 5
http.response_time.2xx:
  min: ..... 0
  max: ..... 16
  mean: ..... 1.7
  median: ..... 2
  p95: ..... 3
  p99: ..... 5
http.responses: ..... 390
vusers.completed: ..... 390
vusers.created: ..... 390
vusers.created_by_name.Rates (/rates): ..... 390
vusers.failed: ..... 0
vusers.session_length:
  min: ..... 2.3
  max: ..... 67.9
  mean: ..... 5.5
  median: ..... 4.5
  p95: ..... 8.6
  p99: ..... 23.3
```

Figura 4: Ejecución de escenario base de Artillery

En el estado inicial de la aplicación, también se presenta un gráfico generado al ejecutar un escenario básico contra el endpoint `exchange`. Este gráfico permite visualizar el comportamiento actual de la aplicación y sirve como punto de referencia para evaluar su evolución a medida que se incorporen nuevas mejoras.

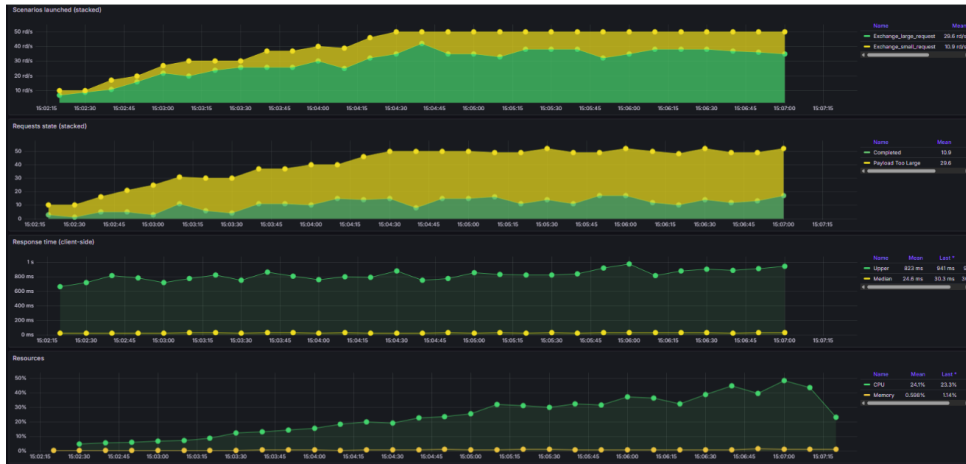


Figura 5: Gráfico de escenario base

6.1. Modularizar la lógica en capas

6.1.1. Hipótesis inicial

Con el objetivo de mejorar la mantenibilidad y reducir el acoplamiento del servicio, se planteó refactorizar la API aplicando una arquitectura simple por capas, sin modificar el comportamiento observable de los endpoints.

En la versión original, la definición de rutas HTTP y parte de la lógica de validación estaban concentradas en `app.js`, mientras que `exchange.js` combinaba responsabilidades de negocio con detalles de acceso al estado persistido en archivos JSON. Esta falta de separación dificultaba la evolución del sistema y aumentaba el impacto de los cambios.

La hipótesis es que una separación en capas (`controller/router`, `servicio` y `repositorio`) permitiría mejorar la modificabilidad del sistema al desacoplar responsabilidades, sin introducir un impacto significativo en la performance ni alterar el comportamiento funcional. No comprometer negativamente la performance es crucial ya que lo consideramos un atributo clave.

6.1.2. Resultado obtenido

El refactor implementó una separación en tres niveles:

- Una capa de **controller/router** que concentra el enrutamiento HTTP y mantiene las mismas rutas, validaciones y códigos de respuesta.
- Una capa de **servicio** que encapsula la lógica de negocio existente.
- Una capa de **repositorio** que abstrae el acceso a la fuente de datos (archivos JSON), dejando preparado un punto de reemplazo para una futura migración a base de datos.

Para este cambio no se elaboró un diagrama de componentes ya que no se vio modificada la arquitectura, pero se decidió realizar un diagrama de módulos para mostrar las capas del refactor.

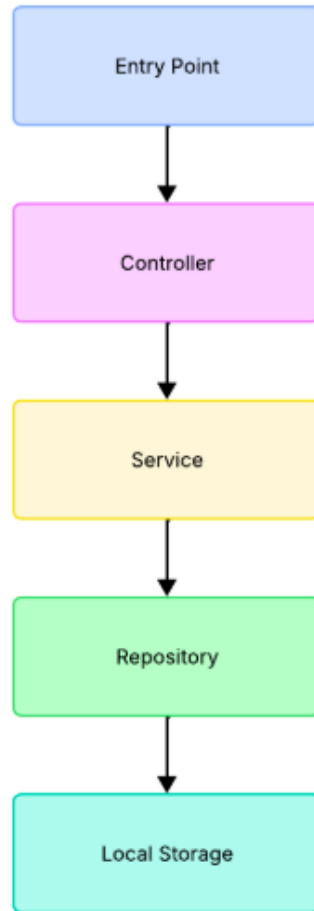


Figura 6: Diagrama de Módulos

El “Entry Point” se trata del archivo “app.json”. Por otro lado, las capas de Controller y Repository se encuentran ubicadas dentro de las carpetas llamadas “controllers” y “repositories” respectivamente. La capa de Service se trata del archivo “exchange.js”, que contiene la lógica de negocio tal cual fue provista por arVault. Finalmente, el Local Storage hace referencia a los archivos json.

Esta reorganización permitió aislar cambios y reducir la propagación de dependencias entre módulos, manteniendo el comportamiento funcional intacto.

Para evaluar el impacto en performance, se ejecutaron pruebas utilizando el script run-scenario.sh con el comando ./run-scenario.sh rates api, que genera carga sobre el endpoint /rates y recolecta métricas. Las pruebas se realizaron en el mismo entorno tanto para la versión original como para la refactorizada.

A continuación se presentan los resúmenes de los resultados obtenidos, mostrando primero el código original y luego el refactor:

```

All VUs finished. Total time: 1 minute, 32 seconds
-----
Summary report @ 18:56:06(-0300)
-----

http.codes.200: ..... 390
http.downloaded_bytes: ..... 42510
http.request_rate: ..... 3/sec
http.requests: ..... 390
http.response_time:
  min: ..... 0
  max: ..... 17
  mean: ..... 2.3
  median: ..... 2
  p95: ..... 4
  p99: ..... 5
http.response_time.2xx:
  min: ..... 0
  max: ..... 17
  mean: ..... 2.3
  median: ..... 2
  p95: ..... 4
  p99: ..... 5
http.responses: ..... 390
vusers.completed: ..... 390
vusers.created: ..... 390
vusers.created_by_name.Rates (/rates): ..... 390
vusers.failed: ..... 0
vusers.session_length:
  min: ..... 1.9
  max: ..... 70.1
  mean: ..... 7.2
  median: ..... 7.2
  p95: ..... 13.9
  p99: ..... 44.3

```

Figura 7: Resultados de performance del código original.

```

All VUs finished. Total time: 1 minute, 32 seconds
-----
Summary report @ 18:42:09(-0300)
-----

http.codes.200: ..... 390
http.downloaded_bytes: ..... 42510
http.request_rate: ..... 3/sec
http.requests: ..... 390
http.response_time:
  min: ..... 0
  max: ..... 27
  mean: ..... 2.1
  median: ..... 2
  p95: ..... 4
  p99: ..... 6
http.response_time.2xx:
  min: ..... 0
  max: ..... 27
  mean: ..... 2.1
  median: ..... 2
  p95: ..... 4
  p99: ..... 6
http.responses: ..... 390
vusers.completed: ..... 390
vusers.created: ..... 390
vusers.created_by_name.Rates (/rates): ..... 390
vusers.failed: ..... 0
vusers.session_length:
  min: ..... 2.1
  max: ..... 56.1
  mean: ..... 6.6
  median: ..... 5.8
  p95: ..... 12.8
  p99: ..... 51.9

```

Figura 8: Resultados de performance tras refactorizar en capas.

Los resultados muestran que la cantidad de respuestas exitosas (código 200) se mantuvo

constante en ambos casos (390 vs 390), al igual que las condiciones de ejecución (cantidad de requests, bytes transferidos y tasa aproximada de 3 req/s), lo que indica que no hubo regresión funcional.

En cuanto a latencia, las métricas principales se mantuvieron estables: $p50 = 2$ ms y $p95 = 4$ ms en ambos casos. En $p99$ se observa una diferencia marginal (5 ms vs 6 ms), atribuible a variabilidad del entorno. Si bien el valor máximo aumentó, esta métrica presenta alta variabilidad y no se considera representativa para el análisis.

6.1.3. Conclusiones del cambio

El refactor permitió mejorar significativamente la modificabilidad del sistema, al introducir una separación clara de responsabilidades. Esto facilita la evolución del código, ya que los cambios pueden realizarse de manera localizada sin afectar otras capas. Un ejemplo concreto de este beneficio se observa en la posterior incorporación de Redis como mecanismo de almacenamiento, la cual requirió modificaciones únicamente en la capa de repositorio.

Como contrapartida, la simplicidad del sistema se ve levemente afectada, ya que la estructura resultante introduce mayor complejidad conceptual y requiere comprender la responsabilidad de cada capa.

En relación a la performance, los resultados empíricos muestran que el costo introducido por la separación en capas es despreciable frente a la latencia del sistema, y no impacta ni en el throughput ni en la tasa de errores. En consecuencia, el trade-off entre simplicidad y modificabilidad resulta favorable en este contexto.

6.2. Migrar el estado a Redis

Como se anticipó en la sección anterior, una vez aplicada la separación por capas se sustituyó la implementación del repositorio basada en archivos JSON por una nueva implementación respaldada por Redis. La API sigue siendo el mismo proceso Node.js/Express, pero todo el estado (cuentas, tasas y log de operaciones) pasa a residir en un servicio externo (`redis:7-alpine` con `-appendonly yes`) accedido por TCP.

6.2.1. Hipótesis inicial

Antes de ejecutar las pruebas, la hipótesis fue que la introducción de Redis no debería degradar de forma significativa la performance percibida por el cliente, a pesar de reemplazar accesos en memoria (orden de microsegundos) por llamadas TCP (orden de milisegundos). El razonamiento fue:

En el escenario de solo lectura (`rates.yaml`), sí se esperaba observar el costo neto del round-trip a Redis, pero en el orden de 1–2 ms, lo que no compromete la experiencia del usuario.

Desde la perspectiva de atributos de calidad, se priorizaron explícitamente escalabilidad, consistencia/integridad y disponibilidad frente a una pérdida controlada en performance y simplicity:

- **Escalabilidad:** extraer el estado del proceso habilita réplicas horizontales de la API detrás de Nginx, eliminando el escalado puramente vertical descrito en el análisis del sistema actual.
- **Consistencia / Integridad:** `hIncrByFloat` es atómico del lado de Redis, lo que reduce la condición de carrera identificada en `exchange()`. Además, con `appendonly yes` se elimina la ventana de hasta 1 segundo de pérdida de datos que introducía el `setInterval/writeFile` del backend JSON.
- **Disponibilidad:** el estado deja de vivir dentro del contenedor de la API; un reinicio del proceso ya no implica reconstruir o leer el último snapshot persistido.
- **Performance:** se elimina el `JSON.stringify(log, null, 2)` periódico de complejidad $O(n)$ sobre el log completo, que en escenarios de larga duración bloquea el event loop. A cambio se paga el costo del roundtrip a Redis.

6.2.2. Resultado obtenido — escenario rates (solo lectura)

El escenario `rates.yaml` mantiene una carga muy baja (3 req/seg promedio) sobre un único endpoint (`GET /rates`). Al no involucrar `/exchange`, expone con mucho menor ruido el costo neto del roundtrip a Redis.

Pre-Redis (backend JSON):

```
http.codes.200: 390
http.request_rate: 3/sec
http.response_time:
min: 1
max: 5
mean: 2
median: 2
p95: 3
p99: 3
vusers.failed: 0
```

Post-Redis:


```
http.codes.200: 390
http.request_rate: 3/sec
http.response_time:
min: 1
max: 5
mean: 3
median: 3
p95: 4
p99: 5
vusers.failed: 0
```

El comportamiento funcional es equivalente (390 vs 390 respuestas exitosas, 0 fallidos) y el throughput se mantiene. La latencia muestra el incremento esperado por el roundtrip TCP: la media pasa de 2 a 3 ms (+1 ms), p95 de 3 a 4 ms (+1 ms) y p99 de 3 a 5 ms (+2 ms). Es un costo absoluto pequeño y conocido (un HGETALL por base de tasas más la agregación final), totalmente aceptable a cambio de los beneficios estructurales descritos.

6.2.3. Análisis cualitativo y trade-offs

Las pruebas confirman que el cambio no degrada significativamente la performance observable, pero el verdadero impacto del cambio se da sobre los atributos que el sistema base no soportaba:

- Scalability: al sacar el estado del proceso, deja de existir el acoplamiento entre estado e instancia que limitaba el sistema a un único nodo. Replicar la API detrás de Nginx ya no produce instancias divergentes.
- Consistencia / Integridad: el uso de `hIncrByFloat` para actualizar balances reemplaza el patrón leer-modificar-escribir en memoria por una operación atómica del lado de Redis, mitigando la condición de carrera identificada originalmente en `exchange()`. La política `appendonly` yes además elimina la ventana de pérdida de datos de hasta 1 segundo que tenía el `scheduleSave` del backend JSON.
- Availability: el reinicio del contenedor de la API ya no implica perder ni reconstruir estado en memoria; el estado vive en un servicio independiente con su propio ciclo de vida y volumen persistente.
- Performance (efecto de segundo orden): se elimina el `JSON.stringify(log, null, 2)` periódico de costo $O(n)$ sobre el log completo, que en ejecuciones largas degrada el event loop de Node.js. Las pruebas duran 1 minuto y por eso no exhiben este efecto, pero arquitectónicamente es una mejora relevante para producción.
- Modifiability: el cambio se concentra exclusivamente en la capa de repositorio. El controller, la capa de servicio (`exchange.js`) y la suite de pruebas Artillery quedan intactos, evidenciando que el refactor por capas previo cumplió su objetivo.

Como contrapartida:

- Simplicity: la topología pasa de un único proceso con archivos locales a dos procesos coordinados por red, lo que agrega un nuevo modo de falla (caída o latencia anómala de Redis). Sin embargo, este modo de falla es operacionalmente conocido y manejable.
- Performance (efecto de primer orden): se introduce un costo de 1–2 ms por operación de estado en el camino de solo lectura, como muestra el escenario `rates`. Es un costo acotado, predecible y asumido conscientemente.

Conclusión: La introducción de Redis se valida como una decisión arquitectónica positiva. Empíricamente, no degrada la performance funcional ni el throughput bajo el mismo patrón de carga utilizado en las iteraciones anteriores, y agrega solamente unos pocos milisegundos de latencia en el camino de solo lectura. A cambio, habilita la escalabilidad horizontal, mejora la consistencia bajo concurrencia, elimina la ventana de pérdida de datos asociada al persistido diferido y reduce el riesgo de bloqueo del event loop por serialización de logs grandes. La separación por capas previa fue la condición que permitió que este cambio se implementara modificando una única capa, sin alterar el comportamiento observable del sistema.

Para esta mejora, la arquitectura sí se vio modificada, así que se realizó un diagrama de componentes y conectores para exponer el cambio.

Diagrama de componentes y conectores:

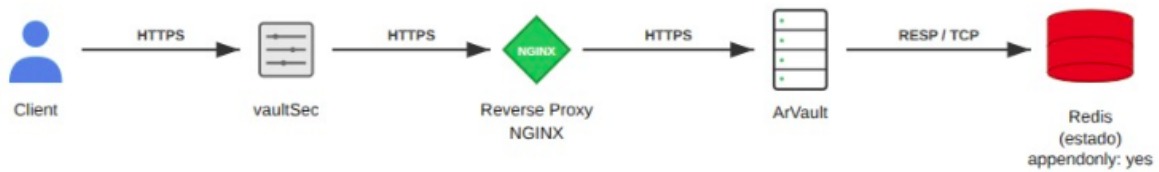


Figura 9: Diagrama de Componentes y Conectores

6.3. Instanciar múltiples instancias del backend

6.3.1. Hipótesis inicial

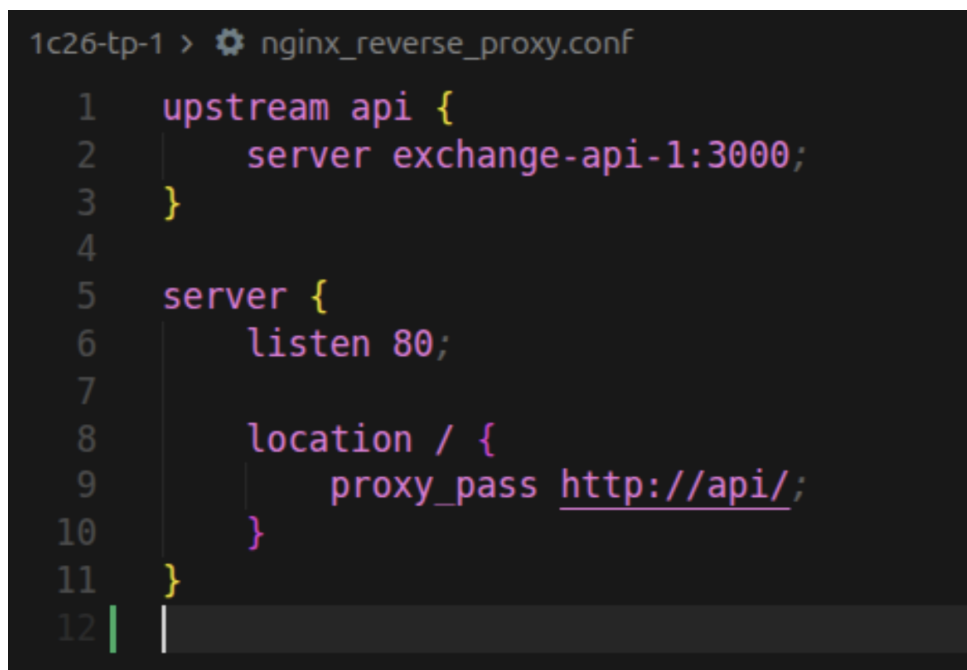
La versión inicial del proyecto contemplaba únicamente una única instancia de backend. En este contexto, el objetivo principal fue optimizar aspectos clave como la Performance, la disponibilidad (Availability) y la Escalabilidad (Scalability) del sistema.

Con el fin de abordar estas limitaciones, se propuso como mejora la posibilidad de ejecutar múltiples instancias del backend, permitiendo así distribuir la carga de trabajo y aumentar la resiliencia de la aplicación ante fallos.

6.3.2. Cambios

Para implementar esta mejora, se efectuaron modificaciones principalmente en el archivo de configuración de NGINX, el cual actúa como punto de entrada y balanceador de carga.

Inicialmente:

A screenshot of a terminal window with a dark background. The prompt is '1c26-tp-1 >'. The file being edited is 'nginx_reverse_proxy.conf'. The configuration is as follows:

```
1 upstream api {
2     server exchange-api-1:3000;
3 }
4
5 server {
6     listen 80;
7
8     location / {
9         proxy_pass http://api/;
10    }
11 }
12 |
```

Figura 10: Configuración inicial de NGINX.

Mejora implementada:

```

1c26-tp-1 > ⚙️ nginx_reverse_proxy.conf
1  server {
2      listen 80;
3      resolver 127.0.0.11 valid=5s ipv6=off;
4
5      location / {
6          set $upstream api;
7          proxy_pass http://$upstream:3000;
8
9          proxy_set_header Host $host;
10         proxy_set_header X-Real-IP $remote_addr;
11         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
12         proxy_set_header X-Forwarded-Proto $scheme;
13     }
14 }

```

Figura 11: Configuración mejorada de NGINX.

Principales cambios:

- Se eliminó el bloque `upstream api`, el cual contenía únicamente un contenedor en su lista (`exchange-api-1`), manteniendo únicamente el bloque `server` en la configuración.
- Al levantar los servicios mediante el comando `docker compose up -d --scale api=N` (siendo `N` la cantidad de instancias), el tráfico deja de ser redireccionado exclusivamente a `exchange-api-1` y pasa a distribuirse hacia el servicio `api` (línea 6). De esta manera, se habilita el uso de múltiples instancias del backend.
- Adicionalmente, se creó un endpoint de prueba llamado `/who`, pensado para validaciones iniciales mediante herramientas como `curl`. Este endpoint permite imprimir el PID del proceso en consola, facilitando la verificación de qué instancia del backend está respondiendo a cada solicitud.

6.3.3. Escenarios

Se creó un escenario nuevo, detallado en `exchange-load.yaml`, que le pega al endpoint `/exchange` a la misma cuenta y con el mismo monto, que tiene simulado un tiempo largo de procesamiento (random entre 400ms y 600ms)

Se comporta de esta manera:

- **Warm-up:** sube de 2 a 10 req/s durante 30s.
- **Saturation ramp:** sube de 10 a 80 req/s durante 120s.
- **Sustained peak:** mantiene 80 req/s durante 90s.

```
1c26-tp-1 > perf > ! exchange-load.yaml
 7  config:
19  phases:
20    - name: Warm-up
21      duration: 30
22      arrivalRate: 2
23      rampTo: 10
24    - name: Saturation ramp
25      duration: 120
26      arrivalRate: 10
27      rampTo: 80
28    - name: Sustained peak
29      duration: 90
30      arrivalRate: 80
31
32  scenarios:
33    - name: Exchange (USD→ARS)
34      flow:
35        - post:
36          url: "/exchange"
37          json:
38            baseCurrency: "USD"
39            counterCurrency: "ARS"
40            baseAmount: 1
41            baseAccountId: 11
42            counterAccountId: 10
43
```

Figura 12: Perfil de carga simulado para el escenario de exchange.

Escenario nuevo ejecutado con un solo backend. Esto fue corrido con el código base entregado por arVault, donde solo se instancia un backend:



Figura 13: Resultados del escenario ejecutado con una única instancia de backend.

Se muestra como hay un tiempo de respuesta promedio de 600ms aproximadamente (amarillo) y un maximo de 800 ms (verde). Tambien se ve que todas las respuestas del servidor son de código 200, lo que indica una buena **Availability**

Mismo escenario ejecutado con 3 instancias levantadas:

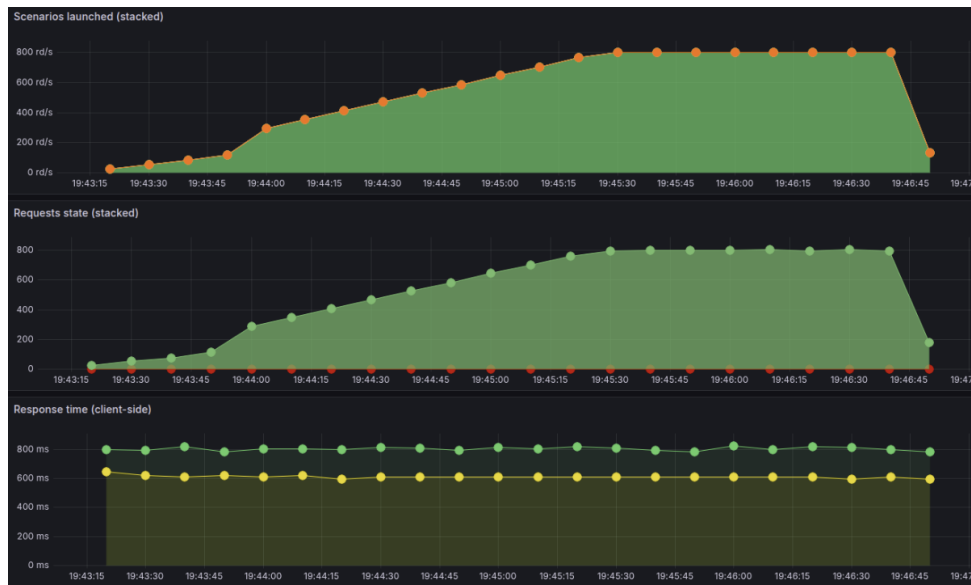


Figura 14: Resultados del escenario ejecutado con 3 instancias de backend.

Se mantienen los tiempos de respuesta en ambos escenarios.

6.3.4. Conclusiones

La hipótesis inicial de que la **Performance** mejoraría al agregar múltiples instancias de backend no se cumple. Esto se debe a que el comportamiento que simula el componente “exchange”

está implementado de forma asincrónica, por lo que no existe un busy wait que genere bloqueo del procesamiento. Como resultado, si bien se observa un aumento en la cantidad de requests procesados, el tiempo promedio de respuesta se mantiene estable independientemente de la cantidad de instancias.

Por otro lado, sí se evidencia una mejora en la **Availability**, ya que al contar con múltiples instancias disminuye la probabilidad de indisponibilidad total del sistema, en comparación con el escenario de una única instancia. Tengo un single point of failure menos en el sistema. Asimismo, se mejora la **Scalability y la Elasticity**, dado que es posible ajustar dinámicamente la cantidad de instancias en ejecución mediante el comando `docker compose up -d --scale api=N`. Esto permite incrementar o reducir el número de contenedores activos según la demanda, logrando adaptarse de forma flexible a las necesidades del sistema en cada momento.

Este cambio impacta negativamente en la aplicación su **Simplicity y Manageability** que implican inherentemente tener más instancias de backend.

También impacta negativamente en su **Security** (hay más puertas de acceso) y su **Testability**, ya que implica tener que usar una suite de testeo que permita simular múltiples instancias si se quiere poder testear sin tener que levantar efectivamente todos los servicios.

6.3.5. Diagrama

Esta mejora se realizó de forma independiente de las mejoras anteriores para evaluar la influencia de múltiples instancias de backend de forma aislada. En la siguiente sección se la integró con el resto de los cambios. Teniendo esto en cuenta, a continuación se muestra un diagrama de Components and Connectors del sistema con esta solución implementada.

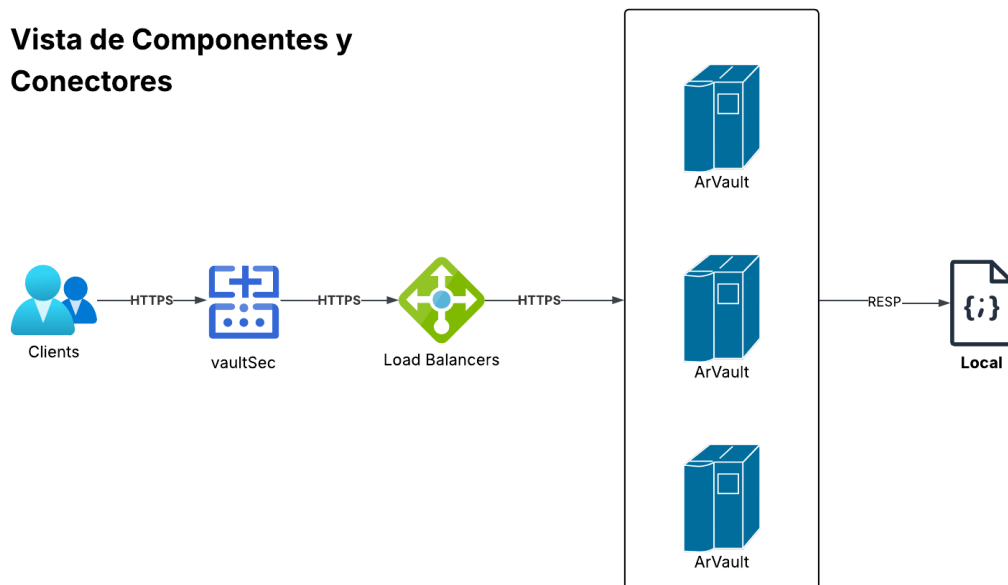


Figura 15: Diagrama de Components and Connectors del cambio introducido

6.4. Limitar cantidad, tamaño de request y tiempo de respuesta

Para la última mejora implementada, se integraron todas las mejoras anteriores para evaluar el sistema en su totalidad. Se realizó una prueba comparativa sobre un servicio de tipo API sometido a carga mixta (requests pequeñas y grandes), con el objetivo de evaluar el impacto de introducir una capa de control mediante NGINX y escalamiento horizontal.

El experimento compara dos escenarios:

- Un escenario base sin control ni balanceo.
- Un escenario mejorado con múltiples instancias y políticas de regulación.

6.4.1. Escenario 1: Arquitectura base

Hipótesis: Se planteó que, si el sistema operaba con una única instancia del servidor y sin ningún tipo de control sobre las requests entrantes (sin limitación de tasa, tamaño o tiempos), entonces:

- El sistema tendría tiempos de respuesta bajos en condiciones favorables
- Pero no sería capaz de manejar carga sostenida
- Y fallaría de forma no controlada ante picos de tráfico

Resultados: Los resultados confirmaron la hipótesis. El sistema mostró tiempos de respuesta muy bajos en promedio, lo que indica que, cuando logra procesar requests, lo hace rápidamente.

Sin embargo, la mayoría de las requests fallaron con errores internos del servidor (código 500). Esto demuestra que el backend no es capaz de absorber la carga y termina colapsando.

En términos prácticos, el sistema responde rápido solo en condiciones ideales, pero se vuelve altamente inestable bajo presión, generando una experiencia poco confiable.

```
Summary report @ 19:13:49(-0300)

http.codes.200: ..... 252
http.codes.500: ..... 1128
http.downloaded_bytes: ..... 20438193
http.request_rate: ..... 3/sec
http.requests: ..... 1380
http.response_time:
  min: ..... 5
  max: ..... 772
  mean: ..... 119.7
  median: ..... 8.9
  p95: ..... 671.9
  p99: ..... 742.6
http.response_time.2xx:
  min: ..... 424
  max: ..... 772
  mean: ..... 608
  median: ..... 608
  p95: ..... 742.6
  p99: ..... 757.6
http.response_time.5xx:
  min: ..... 5
  max: ..... 57
  mean: ..... 9.4
  median: ..... 8.9
  p95: ..... 13.9
  p99: ..... 22
http.responses: ..... 1380
vusers.completed: ..... 1380
vusers.created: ..... 1380
vusers.created_by_name.Exchange large request: ..... 368
vusers.created_by_name.Exchange small request: ..... 1020
vusers.failed: ..... 0
vusers.session_length:
  min: ..... 9.5
  max: ..... 780.2
  mean: ..... 124.5
  median: ..... 15
  p95: ..... 671.9
  p99: ..... 757.6
```

Figura 16: Resultados de la arquitectura base sin control de tráfico.

6.4.2. Escenario 2: Arquitectura mejorada con NGINX

Hipótesis: La hipótesis planteada fue que, partiendo de la infraestructura ya existente con múltiples instancias —la cual había sido implementada previamente— y utilizando las pruebas diseñadas sobre ese escenario, la incorporación de una capa adicional con NGINX permitiría mejorar el comportamiento del sistema bajo carga.

En particular, se asumió que al añadir mecanismos de control sobre el tráfico, como la limitación de peticiones por cliente, la restricción del tamaño de las solicitudes, la configuración de timeouts y el uso de buffering, el sistema podría gestionar de manera más eficiente las requests entrantes.

Bajo este enfoque, se esperaba que el sistema fuera capaz de soportar un mayor volumen de solicitudes de forma estable, reducir los errores críticos asociados a la saturación del backend y mantener un comportamiento más controlado en escenarios de alta demanda, aun cuando esto implica un incremento en la latencia promedio como parte del costo de introducir dichos controles.

```
Summary report @ 18:22:58(-0300)
-----
http.codes.200: ..... 857
http.codes.413: ..... 320
http.codes.500: ..... 193
http.downloaded_bytes: ..... 2460434
http.request_rate: ..... 3/sec
http.requests: ..... 1380
http.response_time:
  min: ..... 2
  max: ..... 800
  mean: ..... 386.2
  median: ..... 539.2
  p95: ..... 727.9
  p99: ..... 772.9
http.response_time.2xx:
  min: ..... 423
  max: ..... 800
  mean: ..... 611.3
  median: ..... 608
  p95: ..... 742.6
  p99: ..... 788.5
http.response_time.4xx:
  min: ..... 2
  max: ..... 15
  mean: ..... 3.6
  median: ..... 3
  p95: ..... 5
  p99: ..... 7.9
http.response_time.5xx:
  min: ..... 6
  max: ..... 33
  mean: ..... 9.1
  median: ..... 7.9
  p95: ..... 13.1
  p99: ..... 16
http.responses: ..... 1380
users.completed: ..... 1380
users.created: ..... 1380
users.created_by_name.Exchange large request: ..... 320
users.created_by_name.Exchange small request: ..... 1060
users.failed: ..... 0
users.session_length:
  min: ..... 6.8
  max: ..... 809.7
  mean: ..... 391.4
  median: ..... 539.2
  p95: ..... 742.6
  p99: ..... 788.5
```

Figura 17: Resultados de la arquitectura mejorada con políticas en NGINX.



Figura 18: Resultados de la arquitectura mejorada con políticas en NGINX.

Como mencionamos anteriormente, en esta sección se está analizando el sistema completamente integrado, por lo que los gráficos mostrados hablan del sistema final.

6.4.3. Resultado obtenido

Los resultados validaron la hipótesis. La cantidad de requests exitosas aumentó significativamente, lo que demuestra una mejora directa en la capacidad del sistema para procesar carga. Además, los errores internos del servidor se redujeron de forma considerable, lo cual indica que el backend dejó de saturarse. Sin embargo, aparecieron errores de tipo 413 (payload demasiado grande). Este cambio es importante, ya que refleja que ahora el sistema no falla de forma descontrolada, sino que aplica políticas explícitas de validación y rechazo. Por otro lado, la latencia aumentó de manera notable. Esto se explica porque el sistema ahora introduce controles (como rate limiting y buffering), lo que agrega tiempo al procesamiento.

6.4.4. Conclusión

En el escenario mejorado, el sistema ya no reacciona de manera desordenada frente a la carga, sino que pasa a tener un comportamiento mucho más controlado. En lugar de intentar procesar todo hasta colapsar, ahora es capaz de regular el tráfico que recibe, repartir el trabajo entre varias instancias y rechazar aquellas solicitudes que no cumplen con las reglas definidas.

En la práctica, esto cambia completamente la forma en que el sistema se comporta: deja de ser frágil e impredecible y se convierte en una solución mucho más sólida. Ahora funciona de manera más estable, sus respuestas son más consistentes y está mejor protegido frente a picos de uso o situaciones anómalas. Además, al distribuir la carga y tener mecanismos de control, queda preparado para crecer y soportar un mayor volumen de usuarios sin comprometer su funcionamiento.

Adicionalmente, estas mejoras ayudan a mejorar la Security del sistema, ya que contribuyen a prevenir posibles ataques maliciosos de denegación de servicios.

6.5. Versión Final del Sistema

Como se mencionó anteriormente, la mejora anterior integra todas las soluciones propuestas a lo largo del informe. La arquitectura resultante se muestra en el siguiente diagrama:

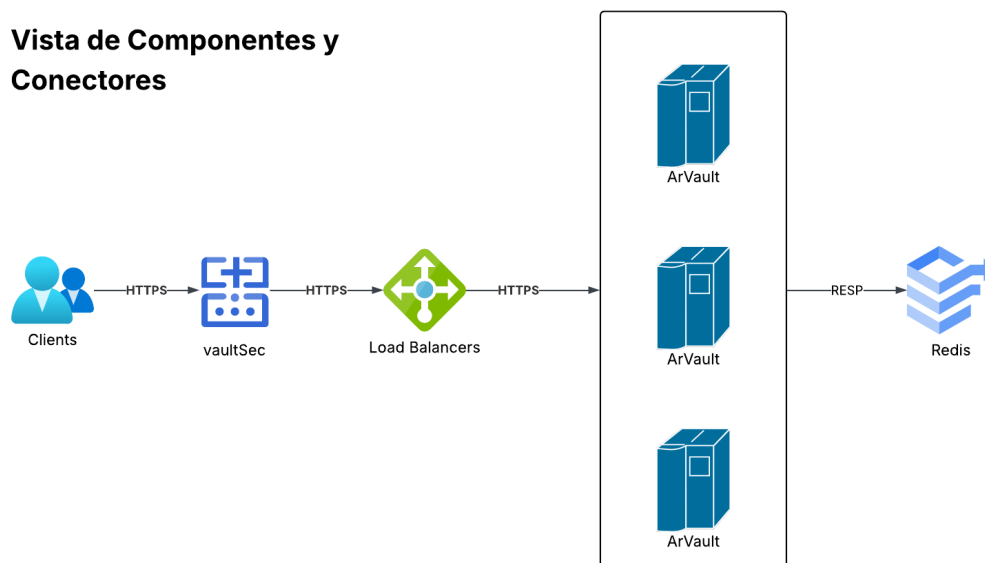


Figura 19: Diagrama de Componentes y Conectores de la arquitectura Final

7. Mediciones Finales y Observabilidad

Para validar el comportamiento del sistema en un entorno productivo simulado, se diseñó un dashboard de observabilidad en Grafana que monitorea en tiempo real el cumplimiento de los **Service Level Objectives (SLO)** definidos para el servicio. La Figura 20 muestra el estado del sistema tras ejecutar un escenario de carga mixta con límites de tamaño de request (`request-size-limit.yaml`).

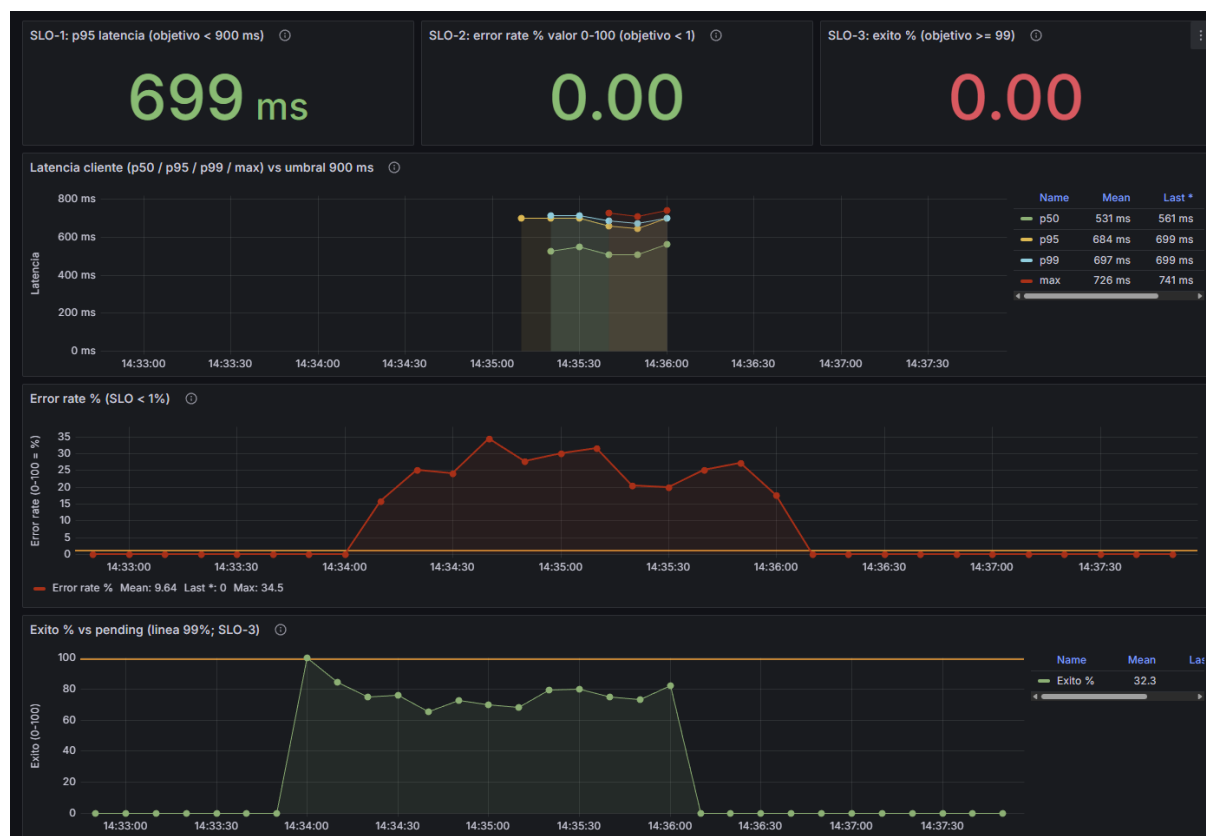


Figura 20: Dashboard de monitoreo de SLOs: Latencia, Tasa de Errores y Éxito.

7.1. Análisis de resultados

El análisis de las métricas obtenidas durante la fase de carga mixta arroja las siguientes observaciones críticas:

- **Gestión de Latencia (SLO-1):** El sistema mantiene una latencia *p95* de **699 ms**, situándose por debajo del objetivo de 900 ms. El gráfico de evolución muestra que, a pesar de los picos de carga, el tiempo de respuesta se estabiliza. Esto confirma que la distribución de carga en múltiples instancias y la persistencia externa en Redis permiten absorber el tráfico sin degradar la experiencia de los usuarios que envían peticiones válidas.
- **Tasa de Error y Filtrado (SLO-2):** Se observa un pico máximo de error del **34.5 %** durante la prueba. Analizando este dato junto con los logs de Artillery, se identifica que estos errores corresponden a respuestas **HTTP 413 (Payload Too Large)**.
- **Disponibilidad y Éxito (SLO-3):** El descenso en este indicador no se debe a una caída del servicio (ya que la latencia sigue siendo baja y no hay errores de conexión), sino a la política de seguridad implementada en NGINX. Al rechazar activamente las requests que exceden el

tamaño permitido, el sistema protege sus recursos, aunque esto se refleje estadísticamente como una disminución en el porcentaje de éxitos sobre el total de intentos.

- **Resiliencia del Sistema:** El gráfico de *Exito % vs pending* muestra cómo el sistema procesa ráfagas de tráfico de manera exitosa (cerca al 100 %) hasta que entran en juego las políticas de restricción. Una vez finalizado el estrés de carga pesada, los indicadores de salud regresan a niveles normales.

Esta visualización confirma que la arquitectura final es capaz de **discriminar tráfico**, protegiendo la integridad del backend mediante el rechazo controlado de peticiones inválidas, manteniendo siempre los tiempos de respuesta para los usuarios legítimos dentro de los parámetros aceptables.

8. Conclusiones

Las decisiones iniciales de diseño del sistema desarrollado por arVault, como el uso de estado en memoria, una arquitectura monolítica y la persistencia diferida, priorizan la rapidez de desarrollo y la simplicidad. Este enfoque funcionaba bien para las etapas iniciales del proyecto y se alineaba con la filosofía de “fake it till you make it”. Sin embargo, estas elecciones introducen riesgos significativos en escenarios productivos, particularmente en términos de consistencia, disponibilidad y escalabilidad, lo que evidencia las limitaciones de este enfoque a medida que el sistema crece.

Incorporar más componentes es una decisión que siempre conlleva algún impacto negativo, los cuales hay que tener en cuenta y asegurarse que los trade-offs en los atributos de calidad valgan la pena. De entrada, agregar más instancias de backend, así como componentes nuevos (como redis) impacta negativamente en la **Simplicity** así como en la **Maintainability**. También, el hecho de tener más nodos suele impactar negativamente en la **Security**. Todo esto se hace a costa de mejorar los atributos mencionados en cada caso.

Es importante poder plantear métricas que nos ayuden a probar (o no) una hipótesis que planteamos previamente a implementar una mejora. Un caso claro de este trabajo fue pensar que agregar más instancias de backend iba a mejorar la **Performance** del sistema (Esto se planteó como caso general, sin tener en cuenta la naturaleza de cómo se comportaba cada endpoint). Decir tan a la ligera algo como “agregar mas instancias de backend mejora inherentemente la performance” es sobresimplificar las cosas, y solo cuando se realizó la implementación se vio que no se lograba lo esperado y se analizó mas a fondo el comportamiento del endpoint “/exchange” (ver seccion 6.1. Instanciar múltiples instancias del backend)

En cuanto a los cambios que se introdujeron al sistema, la incorporación de múltiples instancias de backend mejora la disponibilidad y la escalabilidad del sistema, al reducir puntos únicos de falla y permitir una distribución más eficiente de la carga. No obstante, no se observa un impacto significativo en la latencia, debido a la naturaleza asincrónica del procesamiento lo que evita la aparición de bloqueos activos (busy waiting). Además, se mejora la Scalability y la **Elasticity**; pero se vieron perjudicados la **Simplicity**, **Manageability**, **Security** y **Testability**.

La introducción de control de tráfico mediante NGINX representa un cambio fundamental en el comportamiento del sistema bajo carga. Se pasa de un sistema frágil e impredecible, que falla de manera descontrolada, a uno más robusto y predecible, capaz de regular el flujo de requests y proteger al backend. Este beneficio se logra aceptando un trade-off claro: un aumento en la latencia a cambio de una mejora sustancial en estabilidad y confiabilidad.

La modularización en capas evidencia que es posible mejorar la **modificabilidad** del sistema sin afectar de manera significativa la **performance**. La separación de responsabilidades no solo facilita la evolución del código, sino que también incrementa la **evolucionabilidad** del sistema, al permitir introducir cambios estructurales, como la migración del almacenamiento de manera localizada, reduciendo el impacto sobre el resto del sistema y habilitando su adaptación futura a nuevas tecnologías o requerimientos.

Por otro lado, la migración del estado a Redis resulta clave para habilitar la **escalabilidad horizontal** y mejorar la **consistencia de los datos**. Al externalizar el estado, se eliminan problemas asociados a la memoria local y a la persistencia diferida, reduciendo la probabilidad de pérdida de información y condiciones de carrera. Este cambio introduce un costo de latencia mínimo y controlado, que resulta ampliamente justificable frente a los beneficios obtenidos.

Teniendo todo esto en cuenta, el producto final se asemeja más a algo ya en producción a una escala más grande. El hecho de poder instanciar más backends, junto con un balanceador de cargas configurado para ello es algo que se usa en casi cualquier entorno productivo, así como tener el código en un patrón de layers y delegar el tema de la persistencia a un Redis.

Se podría comparar vaultSec con cualquier servicio de autenticación (Por ejemplo Firebase) y lo que tarda el endpoint “/exchange” podría deberse a depender de un servicio externo para

realizar sus tareas (esto se emula con un tiempo random entre 400 y 800ms).

Todos estos puntos y los mencionados anteriormente en el informe hacen que este servicio se asemeje más a un backend ya puesto en producción en la industria fintech.

A partir de las mejoras implementadas y las métricas obtenidas, se evidencia una mejora en la calidad del software, reflejada en un sistema más estable, consistente y predecible bajo carga, así como en una mayor capacidad para manejar errores y escalar.